



LNDRY CUSTOMER FLUTTER APPLICATION

Completion, API Integration, Workflow and Verification Specification

Developer and Coding-Agent Execution Copy

Repository audited	github.com/adityaaaaaw/lndry
Repository baseline	main @ ba0983ec32db0d500792a3301a3140debb621f19
Product surface	Customer Mobile Application - Android and iOS
Requirements baseline	LNDRY Phase 1 MVP approved workflow, API contract and technical annexure
Audit type	Static source-code and contract audit; no runtime execution claimed
Document date	29 June 2026
Overall requirement-compliant progress	37% complete / 63% pending

CONFIDENTIAL - IMPLEMENTATION AND REMEDIATION BLUEPRINT

Document Control and Use

What this report proves, what it does not prove, and how the developer must use it.

Scope of this document

This specification covers the LNDRY Customer Flutter Application only. It does not define the Vendor Partner application, vendor delivery-employee mode, Admin Operations Dashboard, backend repair, server deployment, EC2/VPS configuration, store publishing or production infrastructure.

Purpose

Convert the existing Flutter UI foundation into the complete, locally testable customer application required by the approved LNDRY Phase 1 workflow. The document is intentionally explicit enough to be supplied to a Flutter developer, Cloud Code, Codex, Claude Code or another coding agent together with the repository and approved UI reference assets.

Source hierarchy

Priority	Source	Controls
1	Signed/approved LNDRY System Workflow Confirmation and Approval	Functional scope, user actions, sequence, roles, inclusions and exclusions.
2	LNDRY Phase 1 API Contract and Endpoint Catalogue	Target endpoint capability, access control, inputs and outputs.
3	LNDRY Platform Requirements and Technical Annexure	Security, Maps, payment, capacity, third-party and acceptance requirements.
4	Approved UI/UX screen images and asset pack supplied by the project owner	Exact visual implementation: layout, spacing, typography, colours, icons, illustrations, copy and screen states.
5	Current GitHub repository	Reusable code foundation only. Existing code never overrides the approved workflow or visual references.

Audit limitations

- The GitHub repository was inspected read-only at the single baseline commit identified on the cover.
- The repository could not be cloned or executed in this environment; no claim is made that flutter pub get, build_runner, flutter analyze, widget tests, Android build or iOS build currently passes.
- No continuous-integration status was available for the inspected commit.
- The approved UI/UX images and complete asset pack were not attached to this audit message. Visual fidelity cannot be scored from code alone; the developer must compare each screen against the owner-supplied references.
- Progress percentages are engineering estimates based on observable code and approved workflow coverage, not time-sheet percentages.

No invention rule

When the code, backend contract, UI reference or asset pack is unclear, do not guess. Pause only the affected task, send the project owner a screenshot or code path, explain the ambiguity and ask for written confirmation by message or call. Continue unrelated tasks that are not blocked.

Contents

Execution map for the complete report.

1. 1. Executive decision and progress report
2. 2. Current repository architecture and reusable foundation
3. 3. Critical blockers and root causes
4. 4. Approved customer workflow - canonical sequence
5. 5. Screen-by-screen gap analysis
6. 6. Canonical domain and state model
7. 7. Data layer, API client and session architecture
8. 8. Authentication and onboarding implementation
9. 9. Customer profile, addresses and Google Maps
10. 10. Home, search, discovery and vendor comparison
11. 11. Vendor detail, garment selection and server quotation
12. 12. Slot capacity, checkout, Razorpay and order creation
13. 13. Order lifecycle, OTP, rejection, cancellation and history
14. 14. Notifications, reviews, invoices and account settings
15. 15. UI/UX and asset fidelity rules
16. 16. Features and code that must be removed or disabled
17. 17. Safe migration and regression-control plan
18. 18. Test strategy and acceptance scenarios
19. 19. Local-development setup and build gates
20. 20. Definition of done and developer sign-off
21. Appendix A. Complete customer endpoint matrix
22. Appendix B. Canonical screen and route inventory
23. Appendix C. Data-model field catalogue
24. Appendix D. Error, empty, loading and offline states
25. Appendix E. Developer/coding-agent execution prompt

1. Executive Decision and Progress Report

Current status of the customer application against the approved Phase 1 MVP.

Official working progress

37% of the approved Customer Application is materially complete. Approximately 63% remains. The existing repository is a strong UI and architecture prototype, but it is not yet a functional LNDRY customer application connected to the approved backend workflow.

Measurement	Estimate	Interpretation
UI screen and navigation shell present	73%	Most major visual screens/routes exist, but screenshot fidelity is unverified and several screens contain placeholders or extra features.
Local mock interaction flow	45%	Some buttons, forms and mock lists respond, but several flows are simulated, inconsistent or broken.
Approved backend/API integration	4%	No ApiCustomerRepository is implemented; the runtime provider always returns MockCustomerRepository.
Google Maps/Razorpay/FCM native integration	5%	Dependencies or production flows are absent or only declared; the visible Maps and payment flows are mock screens.
Requirement-compliant end-to-end customer MVP	37%	Weighted score across architecture, screens, real behavior, API integration, workflow correctness and QA.

Weighted calculation

Area	Weight	Earned	Completion	Reason
Architecture and design-system foundation	10	8	80%	Riverpod, GoRouter, ScreenUtil, feature folders, theme tokens and reusable widgets are present.
UI screens and routing shell	15	11	73%	Major routes exist; visual fidelity and all required states are not proven.
Authentication and session	10	2	20%	Fixed mock OTP, local token and optional Google auth replace the approved backend OTP/session flow.
Profile, address and Maps	10	3	30%	Profile/address screens exist, but Google Maps, Places, exact pin and real address persistence do not.
Discovery, search and vendor selection	12	5	42%	UI exists; eligibility, nearest ranking, search breadth and filters are client-side mock logic.
Garment selection and quotation	12	2	17%	Current cart selects generic services; no garment rate lines, weight quotation or server quote lifecycle.
Slots, checkout, payment and order creation	15	2	13%	Hard-coded slots and mock payment directly create local orders; no hold/draft/Razorpay verification.
Order lifecycle, OTP, history and reviews	13	3	23%	Basic list/detail exists; canonical states, OTPs, rejection recovery, reviews and

Area	Weight	Earned	Completion	Reason
				invoice are missing.
Notifications, devices and realtime refresh	5	1	20%	Notification UI exists, but repository returns an empty list and FCM/device registration is not wired.
Automated verification	3	0	0%	No passing CI/build/test evidence was available.

Safe reuse versus rebuild

Classification	Decision
Keep and adapt	Feature-first folders, shared widgets, ScreenUtil, Riverpod, GoRouter, theme tokens, loading/empty/error components, most approved visual screen layouts.
Replace internally	Repositories, auth state, models, API client, cart/booking state, checkout/payment orchestration, order state handling and hard-coded data.
Remove or disable	Google/Apple social login, coupons/special offers, express surcharge, weekly combo, simulated favorites/share/voice search, customer map-view button, cash/platform-wallet semantics and vendor-facing repository code.
Do not redesign	All supplied UI/UX reference screens and approved assets. Implement same-to-same unless the owner approves a deviation.

2. Current Repository Architecture and Reusable Foundation

Static code inventory of what already exists and should be preserved where correct.

Repository facts

- Public Flutter repository on the main branch with one initial commit.
- Flutter SDK constraint $\geq 3.4.0$ $< 4.0.0$.
- Riverpod state management, GoRouter navigation and ScreenUtil responsiveness are included.
- Dio and secure storage packages are declared, but no production customer API repository is wired.
- Firebase packages are declared, but the customer flow still relies on mock/local authentication.
- Google Maps Flutter and Razorpay Flutter packages are not declared in the inspected pubspec.
- The Android application ID and namespace are still com.example.lndry; release signing uses debug keys. These are deferred release concerns and must not block local functional development.

Current route inventory

Route	Current screen	Assessment
/	Splash	Present; must route using persisted backend session and onboarding state.
/onboarding	Onboarding	Present; visual fidelity must follow supplied reference.
/login	Mobile login	Present; mock/social auth must be replaced with backend OTP only.
/otp	OTP verification	Present; fixed code 1234 must be removed.
/profile-setup	New customer profile	Present; integrate customer profile API.
/location-permission	Location permission	Present; integrate real permission states.
/map-address	Address pin	Present as mock; requires Google Maps, Places, geocoding and API persistence.
/home	Customer home	Substantial UI; replace hard-coded customer/address/category/vendor/order data.
/search	Explore/search	Substantial UI; replace fake services, filters, distance and search.
/cart	Booking basket	Present; remodel from generic service cart to garment quotation basket.
/orders	Orders	Present; wire pagination, active/history states and backend events.
/profile	Profile	Present; wire profile, addresses, preferences and account deletion.
/category/:categoryId	Category vendors	Present; use backend eligible discovery query.
/vendors	Vendor listing	Present; use nearest-first backend ordering and filters.
/vendor/:vendorId	Vendor details	Large UI; use published services, garment rate lines, reviews and slots.
/checkout	Booking review	Present; implement quote, slot hold and prepared order draft.
/checkout/payment	Payment	Mock only; implement Razorpay create/verify/failure/retry.
/checkout/confirmation/:orderId	Post-payment status	Present but wrong message; must show Waiting for Vendor Confirmation.
/orders/details/:orderId	Order tracking	Present but incomplete; add canonical timeline, OTPs, payment and actions.

Current technical strengths

- Feature-first separation makes incremental API migration possible.
- Reusable loading, empty, error, button, card and typography components reduce implementation time.
- Route guards already model onboarding states: unauthenticated, OTP sent, profile required, address required and authenticated.
- AsyncProvider/StateNotifier patterns are already familiar to the codebase.
- Mock JSON files are useful for visual development and deterministic golden tests, but must not remain the production data source.

Architecture risks already present

Risk	Evidence in current code	Required correction
Mock provider is permanent	customerRepositoryProvider directly returns MockCustomerRepository.	Introduce ApiCustomerRepository and environment/provider override; mocks only for tests/storybook.
Retail cart abstraction	CustomerRepository exposes getCart/addToCart and whole-service quantities.	Replace with a booking-draft/quotation state containing vendor, service and garment lines.
Client amount authority	Cart computes platform fee, GST and total using double.	Render server quote amounts in integer paise; never calculate authoritative payable amount in Flutter.
Mixed authentication sources	Firebase/Google and local mock OTP coexist.	Use backend mobile OTP, JWT/refresh session and device registration. Remove social login unless separately approved.
Hard-coded routes/data	Home/search/vendor/checkout contain fixed names, addresses, slots, prices and network images.	Bind every screen to models/API and supplied assets.
Incomplete statuses	Order enum contains only nine generic states.	Implement canonical internal/customer status mapping and immutable timeline events.

3. Critical Blockers and Root Causes

Work that must be corrected before the app can be described as functionally integrated.

ID	Blocker	Current effect	Required result
P0-01	No production customer API layer	The application imports Dio but exposes no ApiCustomerRepository. Every customer provider resolves to local JSON/in-memory mock data.	Build the authenticated Dio client, DTOs, mappers and ApiCustomerRepository; switch development to the local backend URL.
P0-02	Mock OTP and invalid session model	OTP is always 1234, shown on-screen, stored as local mock state and not bound to challenge_id, expiry, attempts, device or refresh token.	Integrate send-otp, verify-otp, refresh-token, session and logout endpoints with secure token storage.
P0-03	Google Maps is only a placeholder	The address screen displays a coloured mock container and saves hard-coded Bengaluru coordinates.	Add Google Maps Flutter, current location, Places search, draggable exact pin, reverse geocoding, serviceability validation and address CRUD.
P0-04	Domain models do not match LNDRY	Service/cart/order models omit garment types, rate units, quote/draft/hold/payment IDs, final quantities, OTP visibility and canonical statuses.	Replace models before wiring APIs; add explicit DTO-to-domain mapping.
P0-05	Garment selection and server quotation missing	Users currently increment whole services rather than shirt/saree/shoe/weight rate lines.	Implement garment-line selection, quantity/weight validation and POST/PATCH quote flow.
P0-06	Capacity-safe slot flow missing	Checkout displays a fixed local slot list and never creates or releases a slot hold.	Load backend slots, hold a slot, show hold expiry, release on exit/failure and prepare order from quote/address/hold.
P0-07	Razorpay is simulated	Payment waits 1.8 seconds, creates a mock order and shows success. No Razorpay dependency or signature verification exists.	Use backend create-order, Razorpay SDK callback, backend verify and idempotent order finalisation.
P0-08	Post-payment state is wrong	Confirmation says Order Confirmed and implies a delivery partner will contact the customer.	Show payment success plus Waiting for Vendor Confirmation; vendor acceptance/rejection happens later.
P0-09	Order tracking and OTPs incomplete	Order detail has seven generic statuses and no pickup/delivery OTP views.	Render immutable timeline, grouped customer labels, purpose-bound OTP cards and visibility/expiry rules.
P0-10	Notifications and device registration not wired	Notification repository returns an empty list and mark methods are no-ops.	Register FCM token, implement notifications/preferences and deep-link order events.
P0-11	Adding a saved address is broken	MapAddressPage calls onboarding-only completeAddressSelection while an authenticated user is adding from Profile; repository.addAddress is not invoked.	Split onboarding default-address creation from authenticated address CRUD and return to the correct calling route.
P0-12	No end-to-end regression proof	No passing build/test/CI evidence is available.	Establish baseline build, unit/widget/golden tests and staged E2E before each migration phase.

High-priority correctness issues

- Home quick-booking silently inserts service svc_001 quantity 2 when the basket is empty. This bypasses the customer-selected vendor, garment lines and server quotation.
- Home contains two-hour pickup examples such as 6-8 PM; the approved workflow uses 60-minute slots.
- Vendor listing defaults to Best Rating, while the approved default is nearest eligible vendors first.
- Distance is generated from the vendor ID hash and is not based on customer/vendor coordinates.
- Search only checks vendor name and description; approved search includes service, category, garment type and vendor name.
- Checkout adds an unapproved 60-minute express pickup surcharge and calls the vendor employee a delivery agent.
- Payment maps net banking to card and exposes cash/platform-wallet concepts that are not part of the approved customer payment model.
- Order confirmation and history do not preserve the selected slot, quote, payment or actual server-created order snapshot.
- Profile falls back to a vendor-like email address in the customer application.

- Network-hosted placeholder avatars and Unsplash images contradict the instruction to use the supplied final assets.

4. Approved Customer Workflow - Canonical Sequence

The exact customer business flow that every route, state and API call must preserve.

Step	Stage	Required application behaviour
1	Launch and session restore	Read onboarding flag; call authenticated session only when tokens exist; refresh once if required; route to login/profile/address/home.
2	Mobile OTP login	Enter mobile number; request challenge; show expiry/resent; verify OTP; securely store access/refresh tokens and customer identity.
3	Profile completion	New customer provides full name, optional email and optional profile photograph.
4	Address selection	Use current location, search or saved address; confirm exact draggable Google Maps pin; complete house/building/road/landmark/city/postal code/label/instructions.
5	Serviceability validation	Backend checks whether at least one approved vendor serves the exact coordinate. Do not fabricate availability in Flutter.
6	Home	Show selected address, search, categories, eligible nearby vendors, available services, active order and previous orders.
7	Discovery	Customer selects category/search/garment; backend returns eligible vendors based on radius, approval, active service, capacity and slot availability.
8	Comparison	Default nearest first; support price low/high, best rating, value for money and available now; customer makes final vendor choice.
9	Vendor/service detail	Show vendor profile, rating, distance, hours, policies, service inclusions/exclusions, completion time and published garment rate lines.
10	Garment declaration	Customer selects clothing items and quantities, or estimated weight for weight-based service. Validate min/max and rate unit.
11	Server quotation	Create quote from vendor, service and garment lines; display server estimate and expiry. Flutter never invents total, fee or tax.
12	Pickup slot	Load only remaining-capacity 60-minute slots. Create temporary hold for the chosen quote/date/slot and display countdown.
13	Prepare order	Backend validates quote, selected vendor, address, slot hold and current eligibility; returns order_draft_id, payable amount and immutable snapshot.
14	Razorpay	Backend creates Razorpay order; Flutter opens SDK; success payload is verified by backend. Failure/cancel returns to a retry-safe state.
15	Order submission	Create/finalise one paid order from the verified draft. Initial business status: Waiting for Vendor Confirmation.
16	Vendor response	Vendor accepts or rejects; timeout auto-rejects. Customer receives notification. No automatic reassignment to another vendor.
17	Rejected recovery	Show reason and refund/payment state; customer explicitly selects another vendor and new slot/quote.
18	Pickup assignment	After acceptance, vendor-owned employee is assigned. Customer sees status, not live GPS.
19	Pickup OTP	At permitted stage, customer sees pickup OTP and expiry; tells it to assigned employee; verified event

Step	Stage	Required application behaviour
		moves pickup forward.
20	At vendor and adjustment	Vendor confirms actual garment count/weight; customer sees original and confirmed lines plus audited final amount/payment consequence.
21	Processing	Customer sees grouped At Partner/Processing plus optional current activity such as Washing, Drying or Ironing.
22	Delivery	Packed order is assigned by vendor; customer sees Out for Delivery without a moving map.
23	Delivery OTP	Customer sees separate delivery OTP; assigned employee verifies; order becomes Delivered.
24	Completion	Show final receipt/invoice, full timeline, final amount and review eligibility; customer rates vendor after delivery.

Canonical customer-facing timeline

Order Submitted -> Waiting for Vendor Confirmation -> Vendor Accepted -> Scheduled -> Pickup Partner Coming -> Pickup OTP Verified -> Picked Up -> At Partner -> Processing -> Packed -> Out for Delivery -> Delivery OTP Verified -> Delivered

Alternative terminal/recovery paths

Waiting for Vendor Confirmation -> Rejected by Vendor -> Customer Selects Another Vendor
 Waiting for Vendor Confirmation -> Auto-Rejected -> Customer Selects Another Vendor
 Allowed early status -> Customer Cancelled -> Refund/Pending Refund/Refunded
 Payment Pending -> Payment Failed or Cancelled -> Retry without duplicate order

Customer tracking rule

The customer application must never show continuous rider movement, a live employee marker or in-app turn-by-turn navigation. It shows order status, timestamps, notifications and OTPs only.

5. Screen-by-Screen Gap Analysis

Current code, approved behavior, implementation action and acceptance evidence for every customer surface.

Screen	Current	Gap	Required action	Acceptance proof
Splash	Present	Session logic is local/Firebase-oriented.	Display supplied splash exactly; initialize storage; determine first launch; restore backend session; route without flicker.	Golden test + new/existing/expired-session routing tests.
Onboarding	Present	Functional completion not proven.	Use supplied images/copy exactly; persist completion; skip after first completion; no network dependency for local assets.	Golden tests for all pages and skip/continue behavior.
Login	Present	Phone UI plus Google/Apple extras.	Keep only approved mobile OTP path unless written change request; send account_type=CUSTOMER; handle validation/rate limits/offline.	API contract test and widget error-state tests.
OTP	Present	Fixed visible 1234 code and 4-digit assumption.	Use challenge_id/expires_in from server; configurable length; countdown; resend cooldown; attempts; masked phone; no OTP logging.	Invalid/expired/resend/rate-limit tests.
Profile setup	Present	Local storage only.	PATCH customer profile; optional photo upload only if approved/reference requires it; route to address.	New-user E2E.
Location permission	Present	Needs native permission integration.	Explain permission; request while-in-use; handle denied/denied forever/service off; permit manual search fallback.	Permission-state widget/integration tests.
Address picker	Mock	No map, Places or persistence.	Google Places autocomplete, current location, map, draggable pin, reverse geocode, exact fields, serviceability and save/default.	Golden + Maps adapter tests + serviceable/unserviceable E2E.
Home	Substantial UI	Hard-coded Aarav/address/avatar, mock vendors/categories/orders, coupons.	Bind to discovery/home response and current profile/address; remove extras; show active order/previous orders from API.	Golden test with supplied reference plus loading/empty/error states.
Explore/Search	Substantial UI	Hard-coded grid/price/images; search and filters simulated.	Use categories/search/suggestions/filters endpoints; debounce; backend sorting; no fake distances.	Search query matrix and pagination tests.
Category vendors	Basic	Client gets all categories and mock vendors.	Use category detail and discovery vendors with lat/long/date/slot/category/garment filter.	Eligibility and empty-state tests.
Vendor listing	Substantial UI	Rating default and hash-based distance.	Default nearest; server pagination, sorting and filter state; display server distance/starting price/slot.	Sort/filter contract tests.
Vendor detail	Large UI	Hard-coded 0.8 km, pickup, express, qualities, combo, favorites/share.	Display only backend profile/services/rates/reviews/policies; implement garment selection; remove unapproved extras.	Golden + vendor/service/rate mapping tests.
Booking basket	Present as cart	Whole-service quantities and local fee calculation.	Represent one vendor + one service + garment rate lines/weight; no cross-vendor cart; show server quote.	Unit tests for garment validation and quote mapping.
Slot selection	Present in checkout	Hard-coded slots and express option.	Load remaining-capacity slots; create hold; expiry countdown; release/reselect; 60-minute only.	Hold expiry, full slot and reselect E2E.
Booking review	Present	Client total and no draft preparation.	Call orders/prepare; render immutable snapshot and payable amount; prevent stale quote/address/slot.	Contract test for all validation errors.
Payment	Mock	Simulated delay; no Razorpay.	Create gateway order, open SDK, handle success/failure/cancel, verify signature, poll status when uncertain.	Razorpay test-mode success/failure/idempotency E2E.

Screen	Current	Gap	Required action	Acceptance proof
Confirmation	Present	Incorrectly says confirmed/pending pickup.	Show payment success and Waiting for Vendor Confirmation; include order number, vendor, amount and notification expectation.	Golden and status assertion.
Orders list	Present	Mock/client filtering only.	Paginated active/history list; status/date filter; rejected/auto-rejected/refund labels; pull-to-refresh.	Pagination and filter tests.
Order detail	Basic	Seven statuses, no timestamps/actions/OTP.	Full snapshot, immutable timeline, payment, garments, quantities/weight, original/final amounts, cancellation/recovery, OTP cards, invoice.	Status-by-status golden and action tests.
Pickup OTP card	Missing	No route/widget/API.	Show only at permitted stage; explicit purpose, expiry, copy disabled if policy requires, refresh/resend rules from server.	Visibility matrix and expiry tests.
Delivery OTP card	Missing	No route/widget/API.	Separate purpose-bound OTP at delivery stage; never reuse pickup OTP.	Visibility matrix and successful delivery E2E.
Notifications	UI present	Mock returns empty, mark no-op.	Paginated notifications, unread badge, mark one/all, deep links, preferences and FCM token.	Push/deep-link and unread-count tests.
Reviews	Vendor tab only/unclear	No customer review repository/actions.	List vendor reviews; delivered-order eligibility; create/update/delete own review per policy.	Eligibility and duplicate-review tests.
Profile	Present	Fallback vendor-like email; local auth data.	Load/update backend profile; correct customer fallback; display verified phone.	Profile update E2E.
Saved addresses	Partial	List/delete only; add flow broken; no edit/default UI.	Full CRUD, edit, default, serviceability and safe delete rules.	CRUD E2E.
Settings	Present	Must be checked against scope.	Keep only approved theme/notification/legal/account actions; add notification preferences and account deletion.	Settings golden and API tests.
Help/Legal	Present	Content source not proven.	Use client-supplied support contact, privacy, terms, refund/cancellation links.	Link and copy approval.

6. Canonical Domain and State Model

Replace retail-style and mock models with LNDRY service-booking entities.

Money rule

Integer paise only

Every API/domain money value must be an integer paise amount. Convert to formatted rupees only in presentation extensions. Do not use double for authoritative amounts, fees, taxes, refunds or totals.

```
typedef MoneyPaise = int;
String formatMoney(int paise) => NumberFormat.currency(locale: "en_IN", symbol: "Rs ").format(paise / 100);
```

Required domain entities

Entity	Minimum required fields
Customer	id, name, phone, email?, photo_url?, account_status, created_at
CustomerAddress	id, label, house, building?, road, landmark?, city, state, postal_code, formatted_address, place_id?, latitude, longitude, instructions?, is_default
LaundryCategory	id, name, description, image_url?, icon?, display_order, is_active
VendorSummary	id, public_name, logo_url?, rating, review_count, distance_km, starting_price_paise, completion_time, next_slot, pickup_available, delivery_available, verified
VendorDetail	summary + description, contact display policy, address area, business hours, working days, policies, images, published services
LaundryService	id, vendor_id, category_id, name, description, inclusions, exclusions, completion_time, image_url?, pickup_available, delivery_available, publication_status
GarmentRateLine	id, service_id, garment_type_id, name, unit: PIECE PAIR KILOGRAM, rate_paise, min_quantity, max_quantity, is_active
GarmentSelection	garment_rate_id, quantity? integer, estimated_weight_kg? decimal string
Quote	id, vendor_id, service_id, lines, subtotal_paise, fees/tax breakdown from server, estimated_total_paise, expires_at, rate_snapshot_version
PickupSlot	id, date, start_at, end_at, remaining_capacity, available
SlotHold	id, quote_id, slot_id, expires_at, status
OrderDraft	id, quote snapshot, address snapshot, slot snapshot, payable_amount_paise, expires_at
Payment	id, gateway_order_id, gateway_payment_id?, amount_paise, currency, status, refund_status, failure_code?
LaundryOrder	id, order_number, customer_id, vendor snapshot, service snapshot, garment snapshots, original/final quantity/weight, quote/draft/payment, slot/address, status, timeline, OTP visibility, amounts, assignments summary, timestamps
OrderTimelineEvent	id, internal_status, customer_label, detail?, actor_type, occurred_at
HandoverOtp	purpose: PICKUP DELIVERY, otp?, masked_purpose, expires_at, visible, verified_at?
Review	id, order_id, vendor_id, vendor_rating, rider_rating?, comment?, created_at, editable_until?
Notification	id, type, title, body, deep_link?, is_read, created_at
NotificationPreferences	push/sms/email category matrix

Canonical internal order status enum

```
enum OrderStatus {
  PAYMENT_PENDING,
  PAYMENT_FAILED,
  WAITING_FOR_VENDOR_CONFIRMATION,
  VENDOR_ACCEPTED,
  PICKUP_ASSIGNED,
  GOING_FOR_PICKUP,
  PICKUP_OTP_VERIFIED,
  PICKED_UP,
  RECEIVED_AT_VENDOR,
  PROCESSING,
  PACKED,
  DELIVERY_ASSIGNED,
  OUT_FOR_DELIVERY,
  DELIVERY_OTP_VERIFIED,
  DELIVERED,
  VENDOR_REJECTED,
  AUTO_REJECTED,
  CUSTOMER_CANCELLED,
  ADMIN_CANCELLED,
  REFUND_PENDING,
  REFUNDED
}
```

Customer-label mapping

Internal status	Customer-facing label	Customer action
PAYMENT_PENDING	Payment Pending	Complete or retry payment.
WAITING_FOR_VENDOR_CONFIRMATION	Waiting for Vendor Confirmation	Wait; cancellation only if policy permits.
VENDOR_ACCEPTED / PICKUP_ASSIGNED	Scheduled	View slot and assignment status.
GOING_FOR_PICKUP	Pickup Partner Coming	Prepare garments and pickup OTP.
PICKUP_OTP_VERIFIED / PICKED_UP	Picked Up	View verification timestamp.
RECEIVED_AT_VENDOR	At Partner	Review confirmed quantities/weight and adjustment.
PROCESSING	Processing	View optional current activity.
PACKED	Packed	Wait for delivery assignment.
OUT_FOR_DELIVERY	Out for Delivery	Prepare delivery OTP.
DELIVERY_OTP_VERIFIED / DELIVERED	Delivered	View final receipt and rate order.
VENDOR_REJECTED	Rejected by Vendor	View reason/refund and select another vendor.
AUTO_REJECTED	Auto-Rejected	View timeout/refund and select another vendor.
CUSTOMER_CANCELLED / ADMIN_CANCELLED	Cancelled	View reason and refund state.
REFUND_PENDING / REFUNDED	Refund Pending / Refunded	View payment/refund detail.

7. Data Layer, API Client and Session Architecture

Implementation structure that lets the UI remain stable while mock logic is replaced safely.

Target folder structure

```
lib/
  core/network/
    api_client.dart
    api_error.dart
    auth_interceptor.dart
    retry_policy.dart
    idempotency.dart
  features/<feature>/
    data/dto/
    data/mappers/
    data/repositories/api_<feature>_repository.dart
    domain/entities/
    domain/repositories/
    presentation/providers/
    presentation/pages/
  shared/session/
    session_store.dart
    session_controller.dart
    device_registration.dart
```

Dio client requirements

- Base URL injected through --dart-define, with local Android emulator default `http://10.0.2.2:<port>/api/v1` and iOS simulator default `http://127.0.0.1:<port>/api/v1`.
- 15-30 second connect/receive timeouts with typed timeout errors.
- Authorization bearer injection from secure storage.
- Automatic single refresh-token attempt after 401; queued refresh so concurrent requests do not create multiple refresh calls.
- Retry only safe GET operations or idempotent requests; never blindly retry payment verification or order creation.
- Idempotency-Key header for slot hold/order prepare/payment/order finalisation where backend requires it.
- Parse the standard success/error envelope, `field_errors` and `request_id`.
- Never log Authorization, refresh token, login OTP, pickup OTP, delivery OTP, Razorpay signature or sensitive addresses in release logs.
- Cancel stale search/autocomplete requests and debounce user input.
- Support multipart upload only for approved profile photo flow.

Repository contracts to implement

Repository	Methods
AuthRepository	sendOtp, verifyOtp, refreshToken, getSession, logout
DeviceRepository	registerDevice, removeDevice
CustomerRepository	getProfile, updateProfile, requestAccountDeletion
AddressRepository	list, create, get, update, delete, setDefault, validateLocation, placeAutocomplete, placeDetails
DiscoveryRepository	getHome, listCategories, getCategory, listVendors, getVendor, getVendorServices, getService, search, suggestions, getFilters
QuoteRepository	createQuote, updateQuote
SlotRepository	getPickupSlots, createHold, releaseHold
CheckoutRepository	prepareOrderDraft
PaymentRepository	createGatewayOrder, verifyPayment, getPayment, listPayments

Repository	Methods
OrderRepository	createOrderFromPaidDraft, listOrders, getActiveOrder, getOrder, getTimeline, getPickupOtp, getDeliveryOtp, cancel, selectAnotherVendor, getInvoice
ReviewRepository	listVendorReviews, getEligibility, create, update, delete
NotificationRepository	list, markRead, markAllRead, getPreferences, updatePreferences

Provider migration rule

```
final customerRepositoryProvider = Provider<CustomerRepository>((ref) {
  if (Env.useMocksForVisualTestsOnly) return MockCustomerRepository();
  return ApiCustomerRepository(ref.watch(apiClientProvider));
});
```

Migration safety

Keep mock repositories only for widget/golden tests and explicit visual-demo builds. Normal local development must talk to the local backend so integration defects are discovered early.

8. Authentication and Onboarding Implementation

Replace fixed OTP and mixed Firebase/social authentication with the approved customer login flow.

Required API sequence

Action	Request	Success state	Failure handling
Send OTP	POST /auth/send-otp { phone, account_type: CUSTOMER }	challenge_id, expires_in; route OTP.	Invalid phone, cooldown, rate limit, service unavailable.
Verify OTP	POST /auth/verify-otp { phone, challenge_id, otp, device }	user, roles, access_token, refresh_token, onboarding state.	Invalid, expired, attempts exhausted, account blocked.
Restore session	GET /auth/session	Profile summary, permissions/role, onboarding state.	401 -> one refresh attempt; then clear session.
Refresh	POST /auth/refresh-token { refresh_token, device_id }	Rotated tokens.	Clear local session when invalid/reused.
Logout	POST /auth/logout	Server session revoked; clear secure storage and device state.	Clear local state even when network logout fails.
Register push device	POST /devices	Device ID/platform/FCM token/app version stored.	Retry later without blocking login.

Remove from customer scope

- Google Sign-In button and Firebase Auth Google flow.
- Apple Sign-In simulated button.
- Visible "Use code: 1234" text.
- Fixed mock code and local hash-based user IDs.
- Mock access token stored as a constant.
- Firebase Auth as session source of truth. Firebase Messaging may remain for push notifications.

Onboarding routing matrix

Server/local state	Destination
First app launch, onboarding incomplete	Onboarding
Onboarding complete, no valid session	Login
OTP verified, profile incomplete	Profile Setup
Profile complete, no address/default location	Location Permission / Address Picker
Valid session and default address	Home
Access token expired, refresh succeeds	Previous intended route
Refresh invalid/revoked	Login with "Session expired" message
Account suspended	Blocked account screen with support action

9. Customer Profile, Addresses and Google Maps

Exact-pin address behavior required before discovery and booking.

Address screen functional requirements

- Search address using server-proxied Google Places autocomplete with a session token.
- Use current location after permission grant; manual search remains available when permission is denied.
- Resolve selected place ID to formatted address, latitude, longitude and postal code.
- Display Google map only while selecting/editing an address, with a fixed center pin or draggable marker matching the supplied UI reference.
- Reverse geocode after the pin stops moving; show loading and retry states.
- Collect house/flat, building, road/area, landmark, city, postal code, label and pickup instructions.
- Validate serviceability before final confirmation; explain unserviceable address without deleting entered fields.
- Persist address through backend and set default when required.
- After confirmation, normal screens show address text; customer tracking never embeds a navigation map.

Current MapAddressPage correction

Current code	Problem	Required implementation
Hard-coded Bengaluru/Karnataka and coordinates	Every user saves the same fake location.	Populate from Places/current pin and user-entered address fields.
Mock map container	No exact pin or Google billing-backed map.	Use <code>google_maps_flutter</code> or approved map component.
Calls <code>completeAddressSelection</code> only	Authenticated add-address flow silently fails.	Onboarding: create + set default + complete onboarding. Profile: create/update repository then pop to address list.
No serviceability call	User can proceed from unsupported area.	POST addresses/validate-location before save/use.
No edit/default controls	Address management incomplete.	Add details/edit/delete/set-default flows with confirmation.

Location permission states

State	UI behavior
Not requested	Show purpose screen and Continue/Enter address manually.
Granted	Detect current coordinates and open map.
Denied once	Explain and offer Try Again / Search manually.
Denied forever	Open App Settings and manual search.
Location service disabled	Open device location settings and manual search.
Maps key/config missing in local dev	Show explicit configuration error; never save fake coordinates.

10. Home, Search, Discovery and Vendor Comparison

Replace visual placeholders and client heuristics with backend eligibility and ranking.

Home response binding

Home component	Required data source	Do not do
Selected address	Default/current address repository	Hard-code Koramangala or Home.
Greeting/profile	Authenticated customer profile	Hard-code Aarav/avatar URL.
Categories	GET /laundry/categories or discovery/home	Hard-code service grids.
Nearby vendors	GET /discovery/home or /discovery/vendors with coordinates	Load all vendors then calculate fake distance.
Active order	GET /orders/active	Derive from local JSON only.
Previous orders	GET /orders with delivered/history filter	Show stale mock entries.
Notifications badge	Notification unread count/device events	Always show a red dot.
Promotions	Not in signed MVP	Show coupons/special offers without change request.

Discovery query requirements

GET /discovery/vendors?lat=<lat>&lng=<lng>&category_id=<id>&date=<YYYY-MM-DD>&slot=<slotId>&sort=<nearest|price_asc|price_desc|rating|value>&garment_type_id=<id>&available_now=<bool>&page=1&limit=20

Sort and filter rules

Control	Required behavior
Default	Nearest eligible vendors first.
Price low/high	Backend compares selected service/garment context, not vendor minimum order.
Best rating	Backend rating with deterministic tie-breaker.
Value for money	Backend-defined stable score; Flutter only displays result/explanation.
Available now	Vendors with active service, configured capacity and remaining applicable slot.
Garment type	Only vendors with active rate for the selected garment type.
Search	Service name, category name, garment type and vendor name.

Current extras to remove

- Voice search snackbar unless a real, approved voice-search requirement is added.
- Map View button for vendor results; the approved customer app does not require a vendor map.
- Favorites/heart behavior unless separately approved.
- Hard-coded Bag Care, Tailoring, Weekly Essentials and other categories/combinations not returned by admin-managed categories.
- Fake “verified partners” count, review count, next pickup and delivery date.
- Hash-derived distance and locally invented value score.

11. Vendor Detail, Garment Selection and Server Quotation

The central LNDRY booking model: services process customer-owned garment lines.

Vendor detail data

- Public laundry name, logo/shop image, description, rating and verified review count.
- Approximate straight-line distance returned by backend.
- Opening/closing times, working days, pickup/delivery availability and public policies.
- Published laundry services with category, inclusions, exclusions and completion time.
- Garment rate lines by piece, pair or kilogram, with min/max quantity.
- Only backend-confirmed available slots and capacity context.

Garment selection behavior

Unit	Input control	Validation	Example
PIECE	Integer stepper	min <= quantity <= max; zero removes line.	Shirt x 3
PAIR	Integer stepper	Whole pairs only.	Shoes x 1 pair
KILOGRAM	Decimal input/stepper	Positive estimate within service range; consistent precision.	Regular clothes approx. 5 kg

Quote API

Operation	Request	Response used by UI
Create	POST /quotes { vendor_id, service_id, garment_lines[] }	quote_id, rate snapshot, line amounts, estimate, expiry, warnings.
Update	PATCH /quotes/:quoteId { garment_lines[] }	Recalculated server estimate and expiry/version.

Booking-basket state

```
BookingSelectionState {
  vendorId; serviceId; List<GarmentSelection> lines; Quote? quote;
  PickupSlot? selectedSlot; SlotHold? hold; CustomerAddress? address;
  OrderDraft? draft; Payment? payment;
}
```

One-vendor invariant

A booking selection belongs to exactly one vendor and one service quotation. When the customer changes vendor or service, clear incompatible garment lines, quote, slot hold and draft after confirmation.

Remove local commercial calculations

- 5% platform fee constant.
- 18% GST constant.
- Locally generated discount/combo savings.
- Locally generated express surcharge.
- Any price derived from minOrderAmount or placeholder service price.
- Any use of double as final payable source of truth.

12. Slot Capacity, Checkout, Razorpay and Order Creation

Capacity-safe and idempotent booking sequence required for real transactions.

Required sequence and data flow

Step	Action	Required behavior
A	Load slots	GET /vendors/:vendorId/pickup-slots using date, service_id and quote_id. Render only remaining capacity.
B	Create hold	POST /slot-holds. Save hold_id/expires_at; start local countdown using server time offset where available.
C	Prepare draft	POST /orders/prepare with quote_id, address_id and hold_id. Backend revalidates all snapshots.
D	Create gateway order	POST /payments/create-order with order_draft_id. Receive Razorpay order ID, amount, currency and key.
E	Open Razorpay	Pass only backend amount/order ID. Never trust a Flutter-calculated amount.
F	Verify	POST /payments/verify with gateway payload and draft ID.
G	Finalise	POST /orders with paid draft/payment ID when contract requires separate submission.
H	Confirm state	Display Waiting for Vendor Confirmation. Refresh order detail or subscribe to updates.

Slot hold UX states

State	UI
Available	Selectable 60-minute slot with exact date/time.
Selected / hold creating	Disable duplicate tap and show progress.
Held	Show countdown and "Reserved temporarily".
Expired	Disable payment, explain expiry, reload slots.
Full during hold	Show SLOT_FULL and retain quote/address.
Released by user	Return to slot selection.
Payment failed/cancelled	Release or let backend expire according to contract; offer retry only with valid hold/draft.

Razorpay callback matrix

Callback/result	App action	Backend confirmation
Success callback	Immediately call verify; show verifying state.	Only backend signature/captured status confirms success.
Failure callback	Show safe failure message/code; retain retryable draft when valid.	GET payment status when ambiguity exists.
User cancelled	Return to review with Cancelled state; no order creation.	Release/expire hold according to policy.
Network lost after gateway success	Do not retry charge. Poll payment/order by IDs.	Backend/webhook is source of truth.

Callback/result	App action	Backend confirmation
Duplicate tap/callback	Disable CTA and reuse idempotency key.	One payment/order only.

Correct confirmation copy

Payment successful - awaiting vendor

Your payment has been verified and the order has been sent to the selected laundry partner. Current status: Waiting for Vendor Confirmation. You will be notified when the vendor accepts, rejects or the request times out.

13. Order Lifecycle, OTP, Rejection, Cancellation and History

Complete post-payment customer experience.

Order detail required sections

- Order number and current customer-facing status.
- Vendor snapshot: name, logo, service and approved contact display.
- Selected service and garment rate snapshot.
- Original declared quantities/estimated weight and vendor-confirmed final values.
- Pickup date, 60-minute slot, address and instructions.
- Payment status, estimated amount, final amount, refund/adjustment state.
- Immutable timeline events with timestamps and optional current processing activity.
- Pickup OTP card only when visible and valid.
- Delivery OTP card only when visible and valid.
- Cancellation button only in allowed states and with policy/reason collection.
- Rejected/auto-rejected recovery action to select another vendor.
- Invoice/receipt after applicable completion.
- Review action only after delivered and eligible.

OTP visibility matrix

OTP	Visible when	Hidden when	UI safety
Pickup	Assigned/going for pickup and backend returns visible=true.	Before assignment, after verified/expired/cancelled.	Purpose label, expiry, warning to share only at handover; never cache after logout.
Delivery	Out for delivery/delivery handover stage and visible=true.	Before delivery assignment, after verified/expired/cancelled.	Separate endpoint/state; never reuse pickup value.

Vendor rejection and timeout

- Show Rejected by Vendor or Auto-Rejected as explicit terminal state, not a generic cancelled label.
- Show rejection reason when supplied and payment/refund state.
- Do not automatically choose another vendor.
- “Select another vendor” starts a new quotation and slot selection while reusing address/garment intent only when backend permits.
- Prevent accidental duplicate charge/order.

Cancellation

Requirement	Behavior
Eligibility	Backend decides whether current status is cancellable.
Reason	Send reason_code and optional comment.
Confirmation	Show consequence, refund policy and irreversible warning.
Result	Refresh order/payment timeline and display refund state.
Offline/timeout	Do not optimistically mark cancelled; reconcile with backend.

History and review

- Order history supports active and past states with pagination and stable ordering.
- Each entry shows vendor, service, final amount, payment status and relevant timestamps.
- Only delivered orders pass review eligibility.

- Review supports vendor rating 1-5, optional employee/rider rating only when the approved contract exposes it, optional comment and update/delete policy.
- Never show a review submission UI for rejected, cancelled or undelivered orders.

14. Notifications, Reviews, Invoices and Account Settings

Supporting customer modules required to complete the workflow.

Push notification integration

- Initialize Firebase Core with valid platform configuration supplied by the client.
- Obtain FCM token and register it with device_id, platform and app version after authentication.
- Handle token refresh and remove stale token on logout when appropriate.
- Handle foreground messages with in-app banner/snackbar and refresh affected query.
- Handle background/terminated deep links to order detail or notifications.
- Do not include OTP or sensitive payment values in push payloads.
- Use Socket.IO only if explicitly chosen for live events; FCM plus refresh is sufficient for the approved status-only customer experience.

Notification centre

Function	Required behavior
List	Paginated, unread filter/type filter, newest first.
Unread count	Drive home/profile badge.
Mark read	Optimistic UI with rollback on failure.
Mark all read	Confirmation optional; update count.
Deep link	Validate route/entity ownership before navigation.
Preferences	Push/SMS/email categories loaded and saved through backend.

Invoice/receipt

- Available only when backend returns it for the applicable order state.
- Open signed URL or authenticated PDF stream using a safe viewer/download action.
- Display final amount and payment/refund data from the backend snapshot.
- Do not generate a client-side invoice from local cart values.

Account settings

- Edit permitted customer profile fields.
- Manage saved addresses and default address.
- Notification preferences.
- Privacy policy, terms, refund/cancellation policy and support contacts supplied by the client.
- Logout all local state safely.
- Account deletion/anonymisation request with confirmation and re-authentication when backend requires it.

15. UI/UX and Asset Fidelity Rules

The existing UI is not permission to invent a new design. Approved references control the final presentation.

Same-to-same instruction

Implement every supplied UI/UX screen and asset exactly as provided: hierarchy, spacing, dimensions, typography, colours, iconography, illustrations, imagery, border radius, shadows, copy, button states, safe areas and responsive behavior. Do not redesign, simplify or replace assets without written approval.

Developer visual workflow

26. Create a screen-reference inventory: screen name, reference image filename, target route, assets and all visible states.
27. Place final local assets in typed asset constants; remove Unsplash and other placeholder network URLs.
28. Build each screen with reusable tokens/components while preserving the exact reference output.
29. Capture golden screenshots at the owner-approved baseline device size and at minimum one small Android and one large device.
30. Compare screenshots pixel-by-pixel or with an approved tolerance; fix spacing/overflow before API work is accepted.
31. After API integration, repeat golden tests for loading, populated, empty, error and long-text states.
32. Send a screenshot and precise question when a reference omits a required state; do not invent the state silently.

Responsive and accessibility requirements

- No horizontal overflow at 320dp width or with large text.
- Respect safe areas, keyboard insets and bottom navigation.
- Use semantic labels for icons and controls; minimum touch target approximately 44-48dp.
- Ensure contrast for text, disabled states, status badges and OTP cards.
- Support screen-reader order and form error association.
- Do not scale text by hard-coded pixel values that break accessibility; use ScreenUtil carefully and test system font scaling.
- All user-generated/vendor text must wrap or truncate intentionally.

Asset policy

Asset type	Rule
Logos/icons	Use supplied vector/PNG assets or approved icon set; no generic replacement.
Vendor/customer placeholders	Use approved local placeholders, not random internet portraits.
Service/category imagery	Use final asset pack or backend URLs; provide deterministic fallback.
Animations/Lottie	Only supplied/approved animations; pause for reduced-motion where possible.
Fonts	Use the exact approved family and weights; bundle font if licensing/offline reliability requires it.
Remote images	CachedNetworkImage for backend images with loading/error placeholders; never hard-code Unsplash URLs.

16. Features and Code That Must Be Removed or Disabled

Strict Phase 1 cleanup so the customer app remains a laundry-service product rather than a generic marketplace.

Feature/code	Current location	Action
Google login	Login screen, auth provider, google_sign_in/Firebase Auth flow	Not in approved customer workflow; remove UI and code unless written change request.
Apple login simulation	Login screen	Placeholder action is misleading; remove.
Visible fixed OTP 1234	OTP screen/auth provider	Security defect; remove.
Special offers/coupons	Home	Not included in signed Phase 1 scope.
Promo codes IRONFREE/DRYCLEAN20	Home	Hard-coded non-functional commerce behavior.
Voice search simulation	Search	Not approved and not functional.
Customer vendor map view	Search/vendor listing	Customer flow uses list-based discovery and exact address map only.
Favorites/share simulation	Vendor detail	No approved persistence/API; remove unless separately approved.
Standard/Express selector and +Rs 49	Vendor detail	Not in signed service model.
60-minute express pickup +Rs 99	Checkout	Unapproved surcharge/behavior; remove.
Weekly Essentials combo	Vendor detail	Unapproved bundle/commercial logic.
Cash payment	Payment model	Approved flow is Razorpay online payment.
Platform wallet semantics	Payment UI/model	Only Razorpay-supported wallet method may appear inside gateway; no LNDRY wallet.
Client-calculated fees/GST	Cart/vendor cart/mock order	Backend quote is authoritative.
VendorRepository/customer app vendor-management code	repositories/abstract and mock files	Not needed by customer application; archive or isolate outside customer build.
Firestore/Firebase Storage/Firebase Auth dependencies	pubspec/code	Remove unless a documented customer feature genuinely uses them; keep Firebase Core/Messaging for FCM.
DevPreviewOverlay in normal builds	main.dart	Gate behind explicit local debug flag; never overlay production/release UI.
Placeholder backend domains	env.dart	Use configured local backend base URL; do not silently call non-existent domains.

Do not remove required “wallet” gateway option incorrectly

The signed workflow allows wallet methods supported by Razorpay. That means the Razorpay checkout may expose supported wallets. It does not approve a separate LNDRY stored-value wallet, wallet balance or wallet ledger.

17. Safe Migration and Regression-Control Plan

How to replace mocks without breaking the already-developed UI.

Non-negotiable working method

- Create a backup tag/branch at the audited baseline before modifications.
- Never rewrite the entire application in one pass.
- Keep page widgets and visual components stable while replacing data contracts underneath.
- Introduce DTOs/mappers and API repositories beside mocks, then switch feature by feature.
- Each phase must pass flutter format, flutter analyze, code generation, unit/widget/golden tests and a local smoke build before merge.
- Use small commits grouped by feature and preserve a rollback point.
- Do not delete mock fixtures until equivalent automated tests exist.
- No server deployment work is included; use local backend and local/test third-party configuration only.

Implementation phases

Phase	Name	Work	Exit gate
0	Baseline and visual freeze	Run current project, record analyzer/build output, capture screenshots, inventory UI references/assets, create branch/tag.	No feature changes.
1	Core network and models	Dio client, error envelope, session storage, canonical money/status/domain models, DTO mappers.	App still uses mocks through adapters; build remains green.
2	Backend OTP and onboarding	Auth repository, token refresh/session, profile setup, device identity; remove social/mock auth.	New/existing customer login E2E passes.
3	Maps and address CRUD	Maps/Places/current location/serviceability and real address repository.	Onboarding and profile address E2E passes.
4	Home/discovery/search	Categories, home, search, filters, nearest-first vendors, vendor detail and reviews list.	No hard-coded business data remains.
5	Garment lines and quotes	Service details, rate units, quantity/weight state, create/update quote.	Server amount displayed exactly.
6	Slots and order preparation	Pickup slots, slot holds/expiry/release, address/quote validation and draft.	Full-slot/expired-hold tests pass.
7	Razorpay and finalisation	SDK, create/verify/status, idempotent order creation and correct confirmation.	Test-mode success/failure/cancel passes.
8	Orders, timeline and OTP	Orders list/detail, realtime refresh, rejection, cancellation, pickup/delivery OTP, invoice.	All canonical status golden/E2E tests pass.
9	Notifications/reviews/settings	FCM/device, notification centre/preferences, review lifecycle, account deletion/legal/support.	Customer supporting modules complete.
10	Regression and acceptance	Full test matrix, screenshot comparison, low-end device performance, clean local builds.	Definition of done signed.

Change isolation rules

- One feature branch per phase or bounded module.
- Provider interfaces change only with adapters for current callers.
- No UI refactor mixed into payment/auth/model migrations unless required to match supplied design.
- Use feature flags only temporarily and remove dead branches after acceptance.
- When backend response differs from the target contract, record the discrepancy; do not patch the UI with undocumented guesses.
- Every mutation disables duplicate submit and reconciles server result before optimistic success.

18. Test Strategy and Acceptance Scenarios

Evidence required before the customer application can be called complete.

Automated test layers

Layer	Required coverage
Unit	Money formatting, DTO mapping, status mapping, validators, quote line rules, search query construction, session refresh queue, error parsing.
Repository/contract	Mock HTTP responses for every endpoint success and documented error; request body/query/header assertions.
Provider/controller	Loading/data/empty/error transitions, cancellation, stale request prevention, retry and cache invalidation.
Widget	All interactive screens, validation, disabled/loading states, long text and accessibility semantics.
Golden	Every supplied UI reference plus loading/empty/error states at baseline and responsive device sizes.
Integration	Secure storage/session, router redirects, Maps adapter, FCM deep links, Razorpay wrapper using test/fake gateway.
End-to-end	Real local backend and test services from login through delivered/review.

Mandatory E2E scenarios

- ☐ New customer OTP -> profile -> exact address -> home.
- ☐ Existing customer session restore and token refresh.
- ☐ Invalid/expired OTP, resend cooldown and rate limit.
- ☐ Location denied/manual search; serviceable and unserviceable addresses.
- ☐ Category/search results show only eligible vendors; nearest default; every sort/filter.
- ☐ Garment pieces, pairs and kilograms; min/max validation; quote update and expiry.
- ☐ Last slot becomes unavailable; app handles SLOT_FULL without losing selection.
- ☐ Slot hold expiry before payment; reselect and continue.
- ☐ Razorpay success creates exactly one order.
- ☐ Razorpay failure, user cancellation and network loss after success.
- ☐ Waiting for Vendor Confirmation then vendor acceptance.
- ☐ Vendor rejection and auto-rejection with explicit select-another-vendor recovery.
- ☐ Pickup OTP hidden -> visible -> verified -> hidden.
- ☐ Vendor confirms different quantity/weight and customer sees final amount/payment impact.
- ☐ Processing stage changes displayed as grouped customer timeline.
- ☐ Delivery OTP hidden -> visible -> verified -> delivered.
- ☐ Allowed cancellation and disallowed cancellation.
- ☐ Notification deep link opens the owned order and marks read.
- ☐ Delivered order review eligibility, submit, edit/delete policy and duplicate prevention.
- ☐ Logout clears tokens, OTPs, address-sensitive cache and query state.

Regression gates

Command/gate	Required result
dart format --set-exit-if-changed .	No formatting differences.

Command/gate	Required result
flutter analyze	Zero errors; warnings reviewed and accepted explicitly.
dart run build_runner build --delete-conflicting-outputs	Completes without conflict.
flutter test	All unit/widget/golden tests pass.
flutter build apk --debug	Local Android build succeeds.
flutter build ios --simulator --no-codesign	Local iOS simulator build succeeds where macOS runner is available.
Manual smoke checklist	All critical paths on at least one Android emulator/device.
Visual review	Supplied UI references approved screen-by-screen.

19. Local-Development Setup and Build Gates

Development-only environment. Production server/deployment work is intentionally excluded.

Required local configuration

```
flutter run \
  --dart-define=FLAVOR=development \
  --dart-define=API_BASE_URL=http://10.0.2.2:3000/api/v1 \
  --dart-define=GOOGLE_MAPS_KEY=<local-restricted-key> \
  --dart-define=RAZORPAY_KEY=<test-key>
```

- Android emulator uses 10.0.2.2 for host-machine localhost; physical device uses the development machine LAN address or approved tunnel.
- iOS simulator normally uses 127.0.0.1 when the backend runs on the same Mac.
- Keep local/test keys out of Git and inject with dart-define or untracked local configuration.
- Use Razorpay test mode only during local development.
- Firebase configuration files are client-provided and remain outside public repositories where required.
- No EC2, VPS, Docker deployment, Nginx, DNS, SSL, CI/CD, backup, production monitoring or store-publishing task belongs to this customer-app development brief.

Package corrections

Package area	Action
Dio/codegen versions	Pin compatible versions; remove any where reproducible builds require it.
Google Maps	Add google_maps_flutter and approved location/Places integration packages or backend proxy adapter.
Razorpay	Add razorpay_flutter and wrap SDK behind PaymentGateway interface.
Firebase	Keep firebase_core and firebase_messaging for push; remove auth/firestore/storage/google_sign_in unless approved use remains.
Connectivity	Use for user messaging only; still handle request failures normally.
Generated files	Run build_runner locally/CI; do not rely on missing generated sources.

Local environment failure behavior

- Backend unavailable: show retryable connection error, never fall back silently to mock data.
- Maps key missing: show developer configuration error in debug; do not save fake pin.
- Razorpay key missing: disable payment and show configuration error in debug.
- Firebase unavailable: app remains usable; push registration retries later, while API notifications can still load.
- Mock mode: available only through an explicit visual-test flag displayed in a debug banner.

20. Definition of Done and Developer Sign-Off

The customer application is complete only when every gate below is demonstrably satisfied.

Functional definition of done

- ☐ Backend mobile OTP login, refresh, session restoration and logout work without fixed codes.
- ☐ New customer profile and exact Google Maps address onboarding work.
- ☐ All address CRUD/default/serviceability flows work.
- ☐ Home/search/categories/discovery use real backend data and nearest-first default.
- ☐ Customer selects the final vendor; no auto-assignment occurs.
- ☐ Garment piece/pair/kg lines create a server quotation and client shows server amounts.
- ☐ Only available 60-minute slots are shown and a capacity-safe hold is used.
- ☐ Prepared order draft and Razorpay test payment work with idempotent finalisation.
- ☐ Post-payment state is Waiting for Vendor Confirmation.
- ☐ Vendor acceptance, rejection and auto-rejection update the customer app correctly.
- ☐ Pickup and delivery OTPs follow purpose/stage/expiry visibility rules.
- ☐ Order timeline, actual quantity/weight, final amount, payment/refund and invoice are displayed.
- ☐ Notifications, preferences, order history and delivered-order reviews work.
- ☐ No continuous live employee map exists in the customer app.
- ☐ Unapproved extras and hard-coded commerce data are removed.
- ☐ Every screen matches supplied UI/UX references and assets.
- ☐ All mandatory tests/builds pass and no mock repository is used in normal local integration mode.

Developer sign-off record

Area	Owner	Evidence/link	Status
UI reference inventory			Not started
Android local build			Not started
iOS simulator build			Not started
API contract tests			Not started
Maps/address E2E			Not started
Razorpay E2E			Not started
Order/OTP E2E			Not started
Golden screenshot approval			Not started
Accessibility review			Not started
Final client walkthrough			Not started

Communication requirement

When any supplied UI reference, asset, business rule, endpoint response or transition is ambiguous, contact the project owner by message or call before implementing an interpretation. Include the route, screenshot, code file, expected behavior and the exact decision needed.

Appendix A - Complete Customer Endpoint Matrix

Target API capability mapped to Flutter screens and state changes.

Method	Endpoint	Screen	Input	Output	UI/state effect
POST	/auth/send-otp	Login	phone, account_type	challenge_id, expires_in	Create OTP state; route OTP.
POST	/auth/verify-otp	OTP	phone, challenge_id, otp, device	user, roles, tokens, onboarding	Store tokens; route by onboarding.
POST	/auth/refresh-token	API interceptor	refresh_token, device_id	rotated tokens	Retry original request once.
POST	/auth/logout	Profile/settings	device_id/refresh token	success	Clear session/cache.
GET	/auth/session	Splash/router	Bearer token	profile, role, onboarding	Restore route.
POST	/devices	Session/FCM	device_id, platform, fcm_token, app_version	device record	Enable push.
DELETE	/devices/:deviceid	Logout/settings	device ID	removed	Remove stale device.
GET	/customer/me	Profile/home	-	customer profile	Bind profile.
PATCH	/customer/me	Edit profile	name, email, photo_url	updated profile	Refresh profile.
DELETE	/customer/account	Account settings	reason, confirmation	accepted/anonymised	Logout and final screen.
GET	/addresses	Address selector	-	saved addresses	List/default.
POST	/addresses	Add address	exact address + lat/lng	created address	Save/select.
GET	/addresses/:id	Address edit	id	address	Populate editor.
PUT	/addresses/:id	Address edit	editable fields	updated address	Refresh.
DELETE	/addresses/:id	Address settings	id	success	Remove safely.
PUT	/addresses/:id/default	Address selector	id	default address	Update current address.
POST	/addresses/validate-location	Address confirmation	lat, lng	serviceable, eligible_vendor_count	Allow/stop confirmation.
GET	/maps/place-autocomplete	Address search	query, session_token, location bias	predictions	Suggestion list.
GET	/maps/place-details/:placeld	Address search	placeld	formatted address, lat/lng, postal code	Move pin/populate.
GET	/laundry/categories	Home/explore	locale/page as supported	categories	Category UI.
GET	/laundry/categories/:id	Category	id	category, garment types	Category context.
GET	/discovery/home	Home	address/lat/lng	categories, vendors, active/previous orders	Single home payload.
GET	/discovery/vendors	Listing	lat/lng/category/date/slot/sort/filter/page	eligible vendors + pagination	Server-ranked list.
GET	/discovery/vendors/:vendorId	Vendor detail	vendorId + coordinate context	public vendor profile	Profile UI.
GET	/discovery/vendors/:vendorId/services	Vendor detail	category/garment filters	published services	Service list.
GET	/discovery/services/:serviceId	Service detail	serviceId	inclusions, garment rates, slots summary	Selection UI.
GET	/search	Search	q, lat, lng	grouped service/category/garment/vendor results	Results.
GET	/search/suggestions	Search input	q	typed suggestions	Autocomplete.
GET	/discovery/filters	Filter sheet	current result context	sorts and garment filters	Build filter UI.
POST	/discovery/value-score	Value sort	result context	ranked vendor IDs/explanation	Optional explanation.
POST	/quotes	Garment selection	vendor_id, service_id, garment_lines	quote_id, estimate, expiry	Display estimate.
PATCH	/quotes/:quoteId	Garment selection	changed lines	recalculated quote	Update estimate.
GET	/vendors/:vendorId/pickup-slots	Slot screen	date, service_id, quote_id	remaining-capacity slots	Select slot.
POST	/slot-holds	Booking confirmation	vendor_id, date, slot_id,	hold_id, expires_at	Start countdown.

Method	Endpoint	Screen	Input	Output	UI/state effect
			quote_id		
DELETE	/slot-holds/:holdId	Back/cancel/timeout	holdId	released	Clear hold.
POST	/orders/prepare	Booking review	quote, address, hold	order_draft_id, payable, snapshot	Freeze review.
POST	/payments/create-order	Payment	order_draft_id	razorpay_order_id, amount, currency, key	Open SDK.
POST	/payments/verify	Payment callback	gateway IDs/signature/draft	verified payment/order state	Finalise/confirm.
GET	/payments/:paymentId	Recovery	paymentId	payment/refund status	Reconcile ambiguity.
GET	/payments/history	Profile/payment history	page/filter	payments	Optional approved history view.
POST	/orders	Post-payment finalisation	order_draft_id, payment_id	order waiting for vendor	Confirmation.
GET	/orders	Orders	status/page/date range	orders + pagination	Active/history list.
GET	/orders/active	Home/order	-	latest active order/timeline/OTP flags	Home tracker.
GET	/orders/:orderId	Order detail	orderId	full order snapshot	Render detail.
GET	/orders/:orderId/timeline	Tracking	orderId	immutable events	Timeline.
GET	/orders/:orderId/pickup-otp	Pickup handover	orderId	OTP/expiry/visibility	OTP card.
GET	/orders/:orderId/delivery-otp	Delivery handover	orderId	OTP/expiry/visibility	OTP card.
POST	/orders/:orderId/cancel	Order detail	reason_code, comment	cancel/refund state	Refresh.
POST	/orders/:orderId/select-another-vendor	Rejected recovery	new_vendor_id, slot/quote context	new attempt	Continue explicitly.
GET	/orders/:orderId/invoice	Completed order	orderId	PDF/signed URL	Open receipt.
GET	/reviews/vendors/:vendorId	Vendor reviews	page	reviews	Reviews tab.
GET	/reviews/eligibility/:orderId	Delivered order	orderId	eligible, reason	Show/hide CTA.
POST	/reviews	Rating	order_id, vendor_rating, optional fields	review	Show saved.
PATCH	/reviews/:reviewId	My review	rating/comment	updated review	Refresh.
DELETE	/reviews/:reviewId	My review	reviewId	deleted	Remove.
GET	/notifications	Notifications	unread/page/type	notifications	List/badge.
PATCH	/notifications/:id/read	Notification tap	id	notification	Mark read.
PATCH	/notifications/read-all	Notifications	-	updated_count	Clear badge.
GET	/notification-preferences	Settings	-	preference matrix	Populate.
PUT	/notification-preferences	Settings	preference matrix	updated preferences	Persist.

Appendix B - Canonical Screen and Route Inventory

Recommended customer application route map. Keep actual route names compatible where practical.

Route	Purpose
/	Splash/session bootstrap
/onboarding	Onboarding
/login	Mobile login
/otp	OTP verification
/profile-setup	New customer profile
/location-permission	Location permission
/addresses/select	Address selector
/addresses/new	Google Maps new address
/addresses/:id/edit	Edit address
/home	Home
/search	Search and suggestions
/categories/:id	Category/vendor results
/vendors	Vendor comparison listing
/vendors/:id	Vendor profile
/services/:id	Service and garment-rate detail
/booking	Garment selection/quote basket
/booking/slots	60-minute slot selection and hold
/booking/review	Prepared order review
/payment	Razorpay checkout
/orders/:id/submitted	Waiting vendor confirmation
/orders	Orders active/history
/orders/:id	Order detail and timeline
/orders/:id/rejected	Rejected/auto-rejected recovery
/orders/:id/review	Review form
/notifications	Notification centre
/profile	Profile
/profile/edit	Edit profile
/profile/addresses	Saved addresses
/profile/preferences	Notification preferences
/profile/legal	Legal/support links
/profile/account	Logout/account deletion

Bottom navigation recommendation

Tab	Decision
Home	Keep.
Explore/Search	Keep.
Book/Booking basket	Keep only if the supplied UI reference includes it; represent booking selection, not retail cart.
Orders	Keep.
Profile	Keep.

Appendix C - Data-Model Field Catalogue

Implementation checklist for DTOs, entities and persistence.

Quote line

- garment_rate_id
- garment_type_id/name snapshot
- unit
- rate_paise
- quantity or estimated_weight_kg
- line_amount_paise
- min/max constraints

Order amounts

- estimated_subtotal_paise
- fee/tax line items from server
- estimated_total_paise
- final_subtotal_paise
- final_total_paise
- paid_paise
- adjustment_paise
- refund_paise
- currency

Order handover

- pickup_assignment summary
- pickup_otp_visible/expires_at
- pickup_verified_at
- delivery_assignment summary
- delivery_otp_visible/expires_at
- delivery_verified_at

Order exception

- vendor_rejection_reason
- auto_rejected_at
- cancellation_reason/code
- refund_status
- failure code/message safe for UI

Order timeline

- event_id
- internal_status
- customer_label
- detail/current activity
- actor type
- occurred_at

Pagination

- page
- limit
- total
- total_pages

API error

- code
- message
- field_errors
- request_id
- safe metadata

Session

- access token
- refresh token
- access expiry
- device_id
- customer summary
- onboarding state

Appendix D - Loading, Empty, Error and Offline States

Every screen must include deterministic non-happy-path behavior matching the supplied visual system.

Surface	Required states
Login	Submitting, invalid phone, OTP cooldown, rate limit, offline, blocked account.
OTP	Countdown, resend, wrong OTP, expired, attempts exhausted, server error.
Address	Permission states, Places loading/none/error, map loading, geocode error, unserviceable, save error.
Home	Skeleton, no serviceable vendors, no active order, backend error, pull-to-refresh.
Search	Initial suggestions, no results, pagination loading, filter no results, request cancelled.
Vendor detail	Vendor unavailable, service unpublished, rates empty, reviews empty, slot unavailable.
Quote	Calculating, expired, rate changed, invalid min/max, vendor became unavailable.
Slots	Loading, no slots, full, hold creating, hold expired, release failure.
Checkout	Draft preparing, stale address/quote/hold, payable changed, duplicate submit disabled.
Payment	Opening gateway, verifying, success, failure, cancelled, uncertain/polling, refunded.
Orders	Empty active/history, pagination, stale event refresh, order not found/ownership denied.
OTP handover	Not yet visible, loading, visible, expired, verified/hidden.
Notifications	Empty, unread, pagination, deep-link invalid, permission disabled.
Reviews	Not eligible with reason, submitting, duplicate, edit window closed.
Profile/account	Update loading/error, account deletion confirmation/re-auth, session expired.

Error copy rule

- Show user-actionable language and preserve entered data.
- Display request ID in expandable support details for server errors, not raw stack traces.
- Map stable codes such as SLOT_FULL, QUOTE_EXPIRED, HOLD_EXPIRED, PAYMENT_FAILED, PERMISSION_DENIED and RATE_LIMIT_EXCEEDED to dedicated recovery actions.
- Never report success until the backend confirms the mutation.

Appendix E - Developer / Coding-Agent Execution Instruction

Copy this section together with the repository, backend contract and UI asset folder.

Execution directive

Treat this document, the signed workflow, API contract, technical annexure and supplied UI reference assets as binding. Preserve approved visual design exactly. Implement in phases, run all gates after each phase and do not add unapproved marketplace features.

You are completing the LNDRY Customer Flutter Application in repository `adityaaaaaw/Lndry`.

Rules:

1. Work only on the customer application. Do not add vendor, vendor-employee/rider or admin dashboards.
2. The signed LNDRY Phase 1 workflow controls functionality. The approved UI/UX images and assets control visual output.
3. Preserve and reuse the existing Flutter UI components where they match the references; replace mock repositories, hard-coded data and incompatible models underneath them.
4. Implement backend mobile OTP, exact Google Maps address, eligible vendor discovery, garment piece/pair/kg quotation, 60-minute capacity-safe slots, Razorpay, Waiting for Vendor Confirmation, complete order timeline, pickup/delivery OTP, notifications, history and reviews.
5. Money is integer paise from the server. The Flutter app is never the source of truth for price, fees, tax, capacity, payment or order status.
6. Remove social login, fixed OTP, coupons, express surcharges, combos, fake map view, favorites/share simulations, cash/platform wallet and other unapproved extras.
7. No continuous live rider map. Vendor-owned employees are visible only as order-assignment context where returned.
8. Use the local backend during development. Do not perform EC2/VPS/server deployment work.
9. Do not silently invent missing UI states or API behavior. Send the project owner a message/call request containing the screen, route, code path, screenshot and exact decision needed.
10. Before every merge run format, analyze, code generation, tests, golden comparison and local debug build.
11. Complete one bounded phase at a time and preserve rollback points.
12. Do not declare completion until the Definition of Done and all mandatory E2E scenarios in this specification pass.

Required first response from developer/agent

- Confirm the baseline commit and create a working branch.
- Report the actual flutter --version, pub get, codegen, analyze, test and debug-build results.
- List missing UI reference files/assets/config keys without inventing replacements.
- Provide a phase-by-phase implementation plan referencing this specification.
- Start with Phase 0 and Phase 1; do not begin payment before canonical models/network/session are accepted.

Final Assessment

Decision for the project owner and development team.

Current status

The repository is a visually substantial Flutter prototype with a reusable architecture, but it remains primarily mock-driven. The customer MVP is assessed at 37% requirement-compliant completion. The remaining 63% is mainly real backend integration, canonical LNDRY domain modelling, Google Maps, capacity-safe quotation/slots, Razorpay, complete order states/OTPs, notifications/reviews and verification.

The development team should not discard the existing UI indiscriminately. The correct strategy is to freeze the approved visual output, replace the mock and retail-style behavior in controlled phases, and prove each feature through API contract tests, golden screenshots and end-to-end scenarios.

No production deployment, server or EC2 work is required by this specification. Completion means a locally integrated, testable customer application that follows the approved workflow and matches the supplied UI/UX assets same-to-same.